

Blockchain_Exploration

本实验的目标是在区块链上获得一些动手实践经验。区块链系统相当复杂，单次实验难以涵盖其所有方面。本实验涵盖以下主题：

- MetaMask、钱包、账户
- 交易与区块、发送交易
- 以太坊节点

这个实验将在 SEED 网络仿真器中展开。这些仿真器文件存储在 Labsetup/emulator_* 文件夹中。对于 AMD64 机器，文件夹名为 emulator_NN；对于 Apple 硅芯片机器，则为 emulator_arm_NN。数字 NN 表示区块链网络上的节点数量。

启动仿真器：选择 emulator_10，进入 emulator 文件夹，并运行以下 docker 命令以构建和启动容器：

```
$ dcbuild # 别名为：docker-compose build
$ dcup # 别名为：docker-compose up
```

EtherView：lab 实现了一个名为 EtherView 的简单 Web 应用程序，以显示区块链上的活动。如果想使用这个应用程序，将浏览器指向 <http://localhost:5000/>。在 Blocks 页面中可以看到新创建的区块和最近的交易。如果没有人发送交易，则区块大多是空的，即不包含任何交易。一旦开始发送交易就可以看到区块。用户可以点击区块和交易来查看它们的详细信息。

Task 1: Setting Up MetaMask Wallet

目前有许多与区块链进行交互的方式，对于一些基础操作，可以使用钱包应用程序来管理密钥、查看账户余额并发送和接收交易。MetaMask 是一款非常流行的用于以太坊的钱包应用程序，它既可以作为浏览器插件使用，也提供了独立的移动应用版本。本实验将使用其浏览器插件版本。

升级 Firefox：SEED 虚拟机里的 Firefox 版本太低，安装 MetaMask 会有问题。先按照以下方法升级 Firefox：

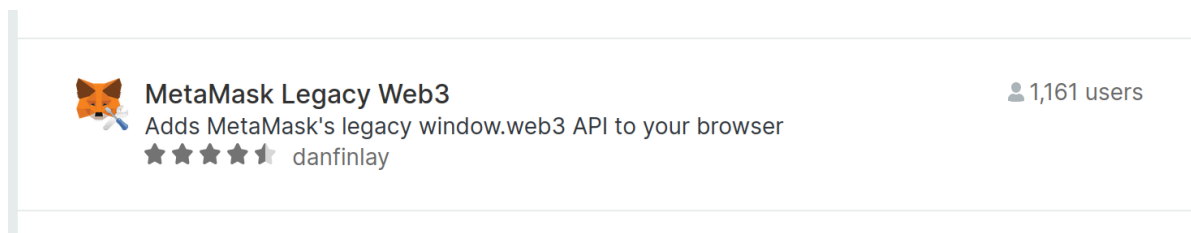
```
$ firefox--version
Mozilla Firefox 83.0
$ sudo apt install firefox
# 升级 Firefox
$ firefox-version
Mozilla Firefox 133.0
```

如下看出更新完成：

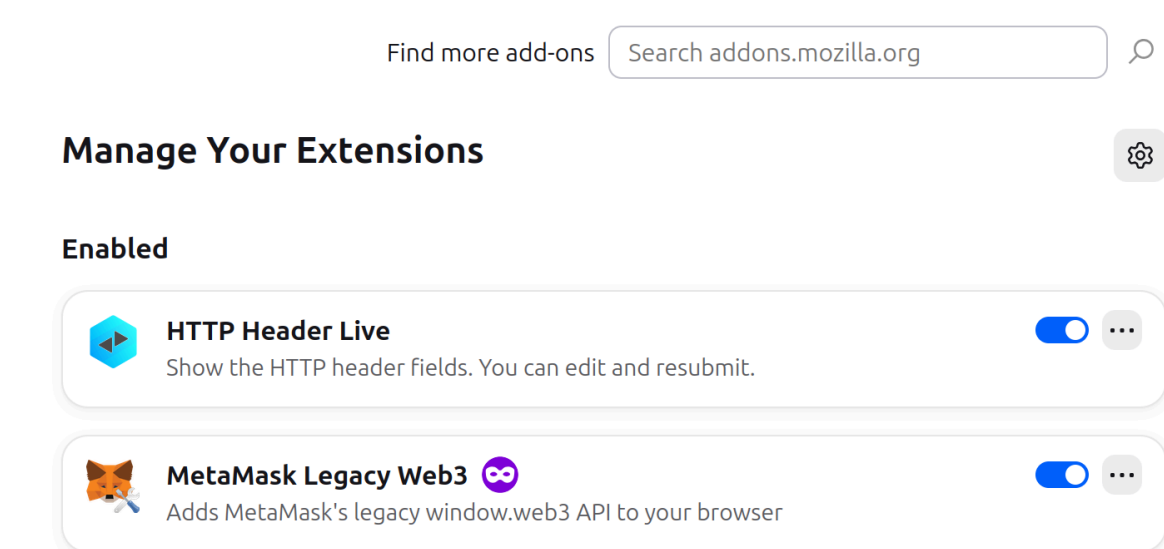
```
firefox-version: command not found
[06/11/26]seed@VM:~/.../emulator_10$ firefox --version
Mozilla Firefox 136.0
```

Task 1.a. Installing the MetaMask extension

进入 Firefox 的菜单页面，点击 "附加组件和主题"，搜索 metamask，找到由 danfinlay 开发的 MetaMask 插件，并按照安装指南进行操作。可以看到应该是下面这个：



不过因为网络问题没办法装插件，配置 clash-for-linux 之后可以装载，结果如下：



Task 1.b. Connecting to the Blockchain

为了使 MetaMask 钱包与区块链建立连接，需要将 MetaMask 连接到区块链网络中的任意一个节点。通过执行 "docker ps" 命令可以检索到所有以太坊节点的 IP 地址，这些地址已经被附加到容器名称上，演示如下：

```
$ docker ps | grep Eth
e372096bb926 as150h-Ethereum-POA-00-Signer-BootNode-10.150.0.71
f0ef91ef9e22 as150h-Ethereum-POA-01-10.150.0.72
3b8c1d191058 as151h-Ethereum-POA-02-Signer-10.151.0.71
...
aea1106d932d as164h-Ethereum-POA-18-Signer-BootNode-10.164.0.71
7cd6fa6888b2 as164h-Ethereum-POA-19-10.164.0.72
```

在虚拟机内使用指令的结果如下，可以看到有非常多的以太坊节点，选择其中的第一个：

[06/11/26]seed@VM:~/.../emulator_10\$ docker ps grep Eth	
0ecd3ef16663 emulator_10_hnode_162_host_0	"/start.sh"
About a minute ago Up 51 seconds	as162h-Ethereum-
POA-07-10.162.0.71	
86fd12bd5879 emulator_10_hnode_151_host_0	"/start.sh"
About a minute ago Up 51 seconds	as151h-Ethereum-
POA-01-10.151.0.71	
88c77b70eb47 emulator_10_hnode_154_host_0	"/start.sh"
About a minute ago Up 52 seconds	as154h-Ethereum-
POA-04-Signer-10.154.0.71	
9e8d80940438 emulator_10_hnode_153_host_0	"/start.sh"
About a minute ago Up 53 seconds	as153h-Ethereum-
POA-03-BootNode-10.153.0.71	
0fc4a6283ee5 emulator_10_hnode_150_host_0	"/start.sh"
About a minute ago Up 52 seconds	as150h-Ethereum-
POA-00-Signer-BootNode-10.150.0.71	
abf42cc7e4ad emulator_10_hnode_152_host_0	"/start.sh"
About a minute ago Up 52 seconds	as152h-Ethereum-
POA-02-Signer-10.152.0.71	
4773ce27a48d emulator_10_hnode_161_host_0	"/start.sh"
About a minute ago Up 52 seconds	as161h-Ethereum-
POA-06-Signer-BootNode-10.161.0.71	

选择一个节点后，需要配置 MetaMask 钱包以便其能连接到该节点。首先进入 MetaMask 的设置菜单，并按照以下步骤操作。需要将 替换为选定节点的实际 IP 地址。

```
Settings > Networks > Add a network > Add a network manually
Network name:
pick any name (e.g., SEED emulator)
New RPC URL:
http://<IP Address>:8545
Chain ID:
1337
Currency symbol: ETH
```

具体填写情况如下：

网络名称

SEED Emulator

根据我们的记录，该网络名称可能与此链 ID 不匹配。

建议名称： [Localhost 8545](#)

默认 RPC（远程过程调用）URL

10.150.0.71:8545

链 ID

1337

货币符号

ETH

可以看到已经正常启用：

代币 去中心化金融 NFT 活动

SEED Emulator



以太坊



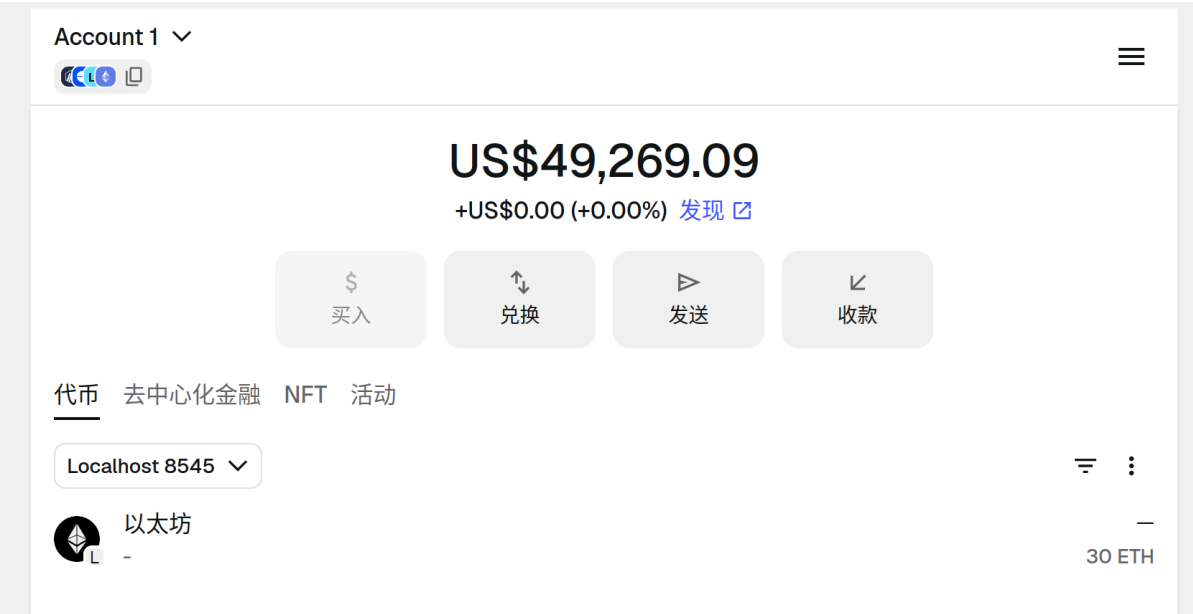
0 ETH

Task 1.c. Adding accounts

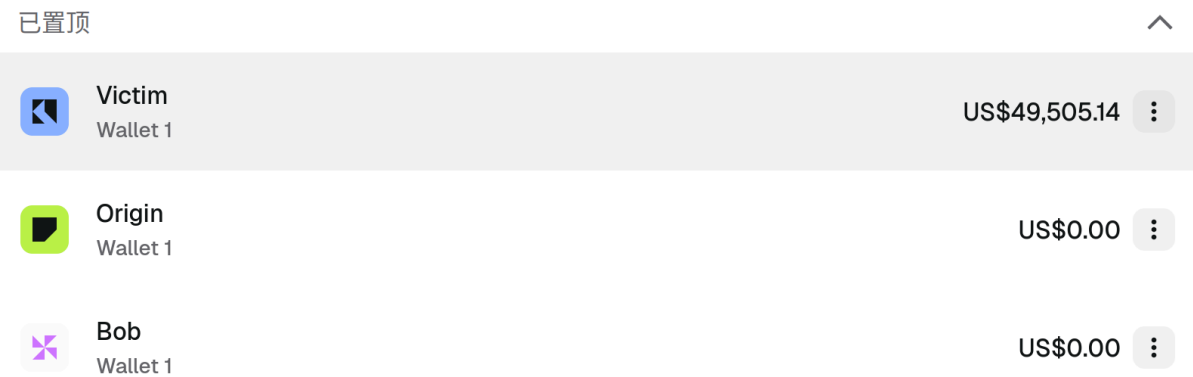
在本任务中将向钱包中添加若干账户，MetaMask 支持创建新的账户或者导入现有账户。在搭建仿真器的过程中已经创建了几个预存资金的账户，这些账户都是基于下述助记词创建的，因此可以通过这些词语恢复：

```
gentle always fun glass foster produce north tail security list example gain
```

把这些已有账户添加至 MetaMask 钱包中，就能使用这些账户来发送交易。为此需要先从 MetaMask 中退出（或锁定账户），回到登录界面。然后点击登录界面的 "Forgot password" 链接，对于 MetaMask 来说，如果忘记了钱包账户的密码，除非之前在其他地方备份了密钥，否则就只能通过使用助记词来恢复密钥，输入之前提供的助记词后，MetaMask 将恢复密钥。MetaMask 会显示区块链上所有余额非零的账户。重新登录后得到如下结果：



可以看到有三个账户，Victim 里面存了 30ETH，即 49,316.80 美元。Origin 和 Bob 都是 0 元。



Task 1.d. Sending transactions

现在可以使用账户来发送交易，请从一个账户向另一个账户转账资金，检查这些账户的余额变化并验证交易是否成功。用 Victim 向 Bob 发送一个 ETH：

<

发送

ETH

至

Bob

Oxab5aa...54890

×

金额

1

ETH ↕

\$1653.25

30 ETH 可用 最大

继续

下图可以看出 Victim 和 Bob 的账户都用正确的变动：

Bob ▾

US\$1,656.48

+US\$0.00 (+0.00%) [发现](#)

\$ 买入

↕ 兑换

▶ 发送

◀ 收款

代币 去中心化金融 NFT 活动

Localhost 8545 ▾

以太坊

-

1 ETH

Victim ▾

US\$48,038.02

+US\$0.00 (+0.00%) [发现](#)

\$ 买入

↕ 兑换

▶ 发送

◀ 收款

代币 去中心化金融 NFT 活动

Localhost 8545 ▾

以太坊

-

29 ETH

Task 2: Interacting with Blockchain Using Python

在本任务中，编写自己的工具来与区块链进行交互，加深对区块链交互机制的理解。所有的操作都在主机虚拟机上执行，本任务中使用的代码存放在 Labsetup/Files 文件夹中。

Task 2.a: Installing Python modules

在 Python 程序中将使用 web3 和 docker 这两个模块，安装命令如下：

```
pip3 install web3==5.31.1 docker
```

换源后下载完成。

Task 2.b: Checking account balance

以下是一段示例代码，展示了如何从区块链中获取账户余额，打开 MetaMask 钱包，查看前三个账户的地址，随后使用这段程序来查询它们的余额，并将程序显示的余额与 MetaMask 钱包显示的余额进行比较。

```
#!/bin/env python3 from web3 import web3
url = 'http://10.150.0.71:8545' web3 = Web3(Web3.HTTPProvider(url)) # Connect to
a blockchain node
addr = web3.toChecksumAddress('0xF5406927254d2dA7F7c28A61191e3Ff1f2400fe9')
balance = web3.eth.get_balance(addr) # Get the balance
print(addr + ": " + str(web3.fromwei(balance, 'ether')) + " ETH")
```

因为后面出问题重装了，变成了三个账户，一个 30 ETH，一个 10 ETH，一个 10.00M ETH。（不知道为什么这么多）只要改变地址查询即可，结果如下：

```
[06/11/26]seed@VM:~/.../Files$ python3 web3_balance.py
/usr/lib/python3/dist-packages/requests/__init__.py:89: RequestsDependencyWarning: urllib3
(2.2.3) or chardet (3.0.4) doesn't match a supported version!
  warnings.warn("urllib3 ({}), or chardet ({}), doesn't match a supported version ")
0xF5406927254d2dA7F7c28A61191e3Ff1f2400fe9: 30 ETH
[06/11/26]seed@VM:~/.../Files$ python3 web3_balance.py
/usr/lib/python3/dist-packages/requests/__init__.py:89: RequestsDependencyWarning: urllib3
(2.2.3) or chardet (3.0.4) doesn't match a supported version!
  warnings.warn("urllib3 ({}), or chardet ({}), doesn't match a supported version ")
0x2e2e3a61daC1A2056d9304F79C168cD16aAa88e9: 9999999 ETH
[06/11/26]seed@VM:~/.../Files$ python3 web3_balance.py
/usr/lib/python3/dist-packages/requests/__init__.py:89: RequestsDependencyWarning: urllib3
(2.2.3) or chardet (3.0.4) doesn't match a supported version!
  warnings.warn("urllib3 ({}), or chardet ({}), doesn't match a supported version ")
0xCBf1e330F0abD5c1ac979CF2B2B874cfD4902E24: 10 ETH
[06/11/26]seed@VM:~/.../Files$
```

可以看到代码查询的数据是符合的。

Task 2.c: Sending transactions

在本任务中编写一个 Python 程序来发送交易，该程序将构建一笔交易，并使用发送者的私钥对其签名，然后通过以太坊节点发送交易。该程序在发送交易后会阻塞，直到交易被确认（也就是交易已经被放在了区块链上）。运行该程序后，检查 MetaMask 中发送者和接收者账户的余额是否发生了变化。可以从 MetaMask 获取私钥，首先点击 "Accountdetails" 菜单，然后点击 "Showprivatekey" 按钮即可。

```
#!/bin/env python3
from web3 import Web3
from eth_account import Account

web3 = Web3(Web3.HTTPProvider('http://ip-address:8545'))

# Sender's private key
key = 'private key'
sender = Account.from_key(key)

recipient = Web3.toChecksumAddress('account number')

tx = {
    'chainId': 1337,
    'nonce': web3.eth.getTransactionCount(sender.address),
    'from': sender.address,
    'to': recipient,
    'value': web3.toWei("11", 'ether'),
    'gas': 200000,
    'maxFeePerGas': web3.toWei('4', 'gwei'),
    'maxPriorityFeePerGas': web3.toWei('3', 'gwei'),
    'data': ''
}

# Sign the transaction and send it out
signed_tx = web3.eth.account.sign_transaction(tx, sender.key)
tx_hash = web3.eth.sendRawTransaction(signed_tx.rawTransaction)

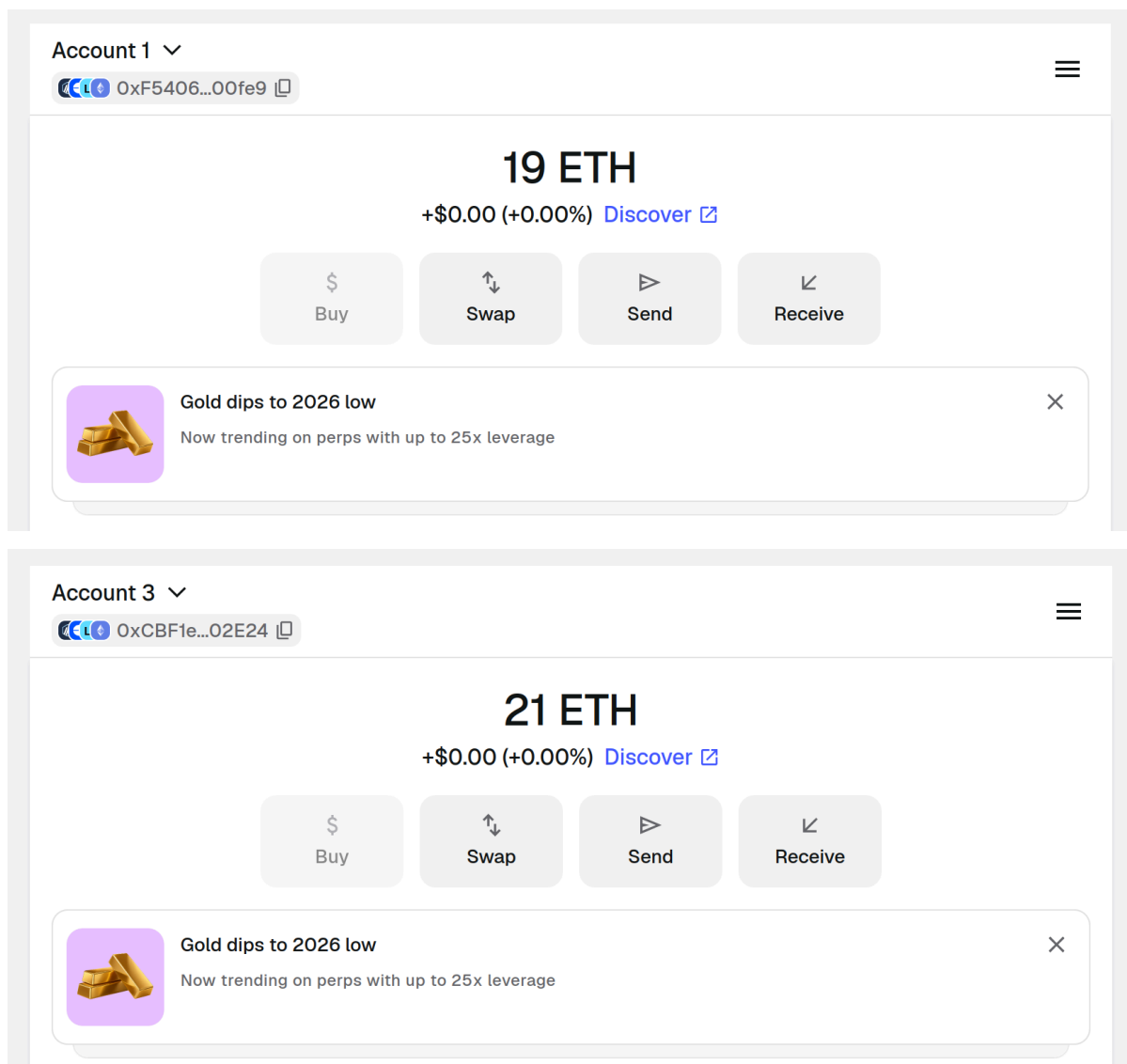
# Wait for the transaction to appear on the blockchain
print("Transaction sent, waiting for receipt ...")
```

按 PDF 填写空位，首先要获得连接的 URL 来确定在哪个区块链上面进行交易，接着填写发送方密钥好使用账户进行交易，最后填写接收方地址好正常完成财产转移。运行结果如下：

[illegible]

Recent Blocks	Recent Transactions
2051 : 0xd28ed1...f80e59 (0)	2049: 0xd20d...1122 (0xF54...fe --> 0xCBF...E2)
2050 : 0xc3185d...ef780b (0)	
2049 : 0x35015a...ca8c82 (1)	

可以看到交易记录成功，转移了 11 ETH，数据也符合：



Task 3: Interacting with Blockchain Using Geth

可以直接通过一个区块链节点与区块链进行交互。在我仿真器中，每个以太坊节点都运行Geth 客户端，这是用 Go 语言实现的以太坊。与 Geth 客户端交互有多种方式，包括 websockets、HTTP 和本地 IPC。当使用 MetaMask 或 Python 程序与 Geth 节点交互时，采用的是 JSON-RPC 方法。此外，还可以登录到 Geth 节点，并使用本地 IPC 与其通信。以下是一个 geth 命令，用于在节点上获得一个交互式控制台：

```
root@f6fb88f9e09d / # geth attach
welcome to the Geth JavaScript console!

instance: Geth/NODE_8/v1.10.26-stable-e5eb32ac/linux-amd64/go1.18.10 coinbase:
0xa888497f7938825f80f35867a1e707f42b9b347d
...
To exit, press ctrl-d
>
```

这是一个交互式的 JavaScript 控制台，可以在其中执行 JavaScript 代码。eth 类提供了丰富的 API 接口，可以方便地与区块链进行交互。以下是一个示例，展示了如何利用这些 API 来查询账户余额：


```
> myaccount = "0xc20ab9a1ab88c9fae8305b302836ee7734c6afbe"
> eth.getBalance(myaccount)
100000000
```

Task 3.a: Getting balance

从 MetaMask 钱包中获取前三个账户的余额，并查看结果是否与 MetaMask 上显示的一致。先进入对应容器，再输入上述代码进入 Geth 控制台。按照代码分别填入三个账户的地址 ID，分别是：

```
> myaccount = "0xF5406927254d2dA7F7c28A61191e3Ff1f2400fe9"
> myaccount = "0x2e2e3a61daC1A2056d9304F79C168cD16aAa88e9"
> myaccount = "0xCBf1e330F0abD5c1ac979CF2B2B874cfD4902E24"
```

运行结果如下：

```
> myaccount = "0xF5406927254d2dA7F7c28A61191e3Ff1f2400fe9"
"0xF5406927254d2dA7F7c28A61191e3Ff1f2400fe9"
> eth.getBalance(myaccount)
18999936999999853000
> myaccount = "0x2e2e3a61daC1A2056d9304F79C168cD16aAa88e9"
"0x2e2e3a61daC1A2056d9304F79C168cD16aAa88e9"
> eth.getBalance(myaccount)
9.999999e+24
> myaccount = "0xCBf1e330F0abD5c1ac979CF2B2B874cfD4902E24"
"0xCBf1e330F0abD5c1ac979CF2B2B874cfD4902E24"
> eth.getBalance(myaccount)
21000000000000000000
>
```

基本符合在 19, 10M, 21 附近，查询的数值正确。

Task 3.b: Sending transactions

每个节点都维护着一组账户，这些账户信息存储在 /root/.ethereum/keystore 下，它们的地址被加载到 eth.accounts[] 数组中。例如，如果想要获取数组中第一个账户地址，可以通过 eth.accounts[0] 来获得。这些账户默认是锁定状态的（即使用密码加密），因此在使用这些账户进行交易之前，我们需要先对它们解锁。在我们的仿真器环境中，所有账户都可以使用固定密码 admin 来解锁。

```
> eth.accounts
["0xa888497f7938825f80f35867a1e707f42b9b347d"]
> personal.unlockAccount(eth.accounts[0], "admin")
true
```

现在可以从这些账户向 MetaMask 钱包中的账户转账资金了，请按照下面的示例操作，向 MetaMask 钱包中的一个账户转账，并查看这笔交易的结果是否显示在 MetaMask 上。

```
> sender = eth.accounts[0]
> target = "0xF5406927254d2dA7F7c28A61191e3Ff1f2400fe9"
> amount = web3.toWei(0.2, "ether")
> eth.sendTransaction({from: sender, to: target, value: amount})
"0x8c6c57d5a32de...7304"
```

按照代码先解锁账户，如下：

```
> eth.accounts
["0xa888497f7938825f80f35867a1e707f42b9b347d", "0xce0624bb973438a89e81b86cf68a0dbb167372a5"]
> personal.unlockAccount(eth.accounts[0], "admin")
true
```

然后根据代码设置发送端和接收端，把解锁的账户的财产发到我们的第一个账户上，设置为发送0.2 ETH：

```
> sender = eth.accounts[0]
"0xa888497f7938825f80f35867a1e707f42b9b347d"
> target = "0xF5406927254d2dA7F7c28A61191e3Ff1f2400fe9"
"0xF5406927254d2dA7F7c28A61191e3Ff1f2400fe9"
> amount = web3.toWei(0.2, "ether")
"2000000000000000000"
> eth.sendTransaction ({from: sender, to: target, value: amount})
"0x88ccabee849926b50ae892bc66c4e84aed9cdedeb2b749178818da8d475fb55c"
> █
```

可以看到有交易记录，同时账户内容更改：

Recent Blocks	Recent Transactions
2142 : 0xac6c93...53855e (0)	2141: 0x88cc...fb55 (0xa88...47 --> 0xF54...fe)
2141 : 0xf04685...9d1009 (1)	2049: 0xd20d...1122 (0xF54...fe --> 0xCBF...E2)
2140 : 0x3673ca...a90793 (0)	

Account 1 ▾

 0xF5406...00fe9 

19.2 ETH

+\$0.00 (+0.00%) [Discover](#) 

 Buy

 Swap

 Send

 Receive



Gold dips to 2026 low

Now trending on perps with up to 25x leverage



Task 3.c: Sending transactions from a different account

现在不使用 eth.accounts[0]，而是使用 MetaMask 钱包中的任意一个账户发送交易。即设置 sender 为钱包中的某个账户，请尝试进行交易。让账户 3 往账户 1 进行交易，结果如下：

```
> sender = "0xCBf1e330F0abD5c1ac979CF2B2B874cfD4902E24"
"0xCBf1e330F0abD5c1ac979CF2B2B874cfD4902E24"
> target = "0xF5406927254d2dA7F7c28A61191e3Ff1f2400fe9"
"0xF5406927254d2dA7F7c28A61191e3Ff1f2400fe9"
> amount = web3.toWei(0.2, "ether")
"200000000000000000"
> eth.sendTransaction ({from: sender, to: target, value: amount})
Error: unknown account
    at web3.js:6365:9(45)
    at send (web3.js:5099:62(34))
    at <eval>:1:21(10)
```

而在 Geth 控制台中发送交易时，发送方的私钥必须在节点本地。Geth 不认识发送的账户，所以报错。

Task 4: Adding a Full Node

在本任务中将学习如何将一个新的节点加入到现有的区块链网络中。已经准备了一个名为 `new_eth_node` 的空容器，任务是对这个容器进行配置，使其成为一个完备的以太坊节点。首先需要使用区块链的初始信息初始化节点，这些信息被保存在创世区块中 (genesisblock)，创世区块是区块链的第一个区块，可以从仿真器中以以太坊节点的 `/tmp/eth-genesis.json` 文件中找到创世区块的内容：

```
geth--datadir /root/.ethereum init /eth-genesis.js
```

然后执行 `geth` 命令将新节点加入到现有的区块链网络中。为此需要提供一份引导节点列表，可以在任何现有的以太坊节点（非引导节点）的 `/tmp` 目录下找到一个名为 `eth-node-urls` 的文件。该文件包含了区块链网络上的所有引导节点的信息。需要将这个文件的内容复制并粘贴到 `new_eth_node` 容器中的 `/tmp/eth-node-urls` 文件中。在执行以下的 `geth` 命令时，将使用 `/tmp/eth-node-urls` 文件中的内容作为 `bootnodes` 选项的参数，以便新节点能够连接到网络中的其他节点：

```
geth --datadir /root/.ethereum --identity="NEW_NODE_01" --networkid=1337 \
  --syncmode full --snapshot=False --verbosity=2 --port 30303 \
  --bootnodes "$(cat /tmp/eth-node-urls)" --allow-insecure-unlock \
  --http --http.addr 0.0.0.0 --http.corsdomain "*" \
  --http.api web3,eth,debug,personal,net,clique,engine,admin,txpool
```

根据上述步骤将 `new_eth_node` 容器配置成以太坊节点。先找到空容器：

```
[06/12/26]seed@VM:~/.../Files$ docker ps -a | grep new_eth_node
25f8b260094b      emulator_10_hnode_150_new_eth_node      "/start.sh"
10 hours ago      Up 3 hours                                as150h-new_e
th_node-10.150.0.74
```

然后找到一个已有的容器，找到文件并复制：

```
[06/12/26]seed@VM:~/.../Files$ docker exec -it 0fc4a6283ee5 /bin/bash
root@0fc4a6283ee5 / # cat /tmp/eth-node-urls
enode://763815de0d0751de72cf136df6139cb656f7c32ea07988455c9fa52e0f01544ba10e5d249ee5d1e417a
a4b542d5d0e15e9da03cea0b3f821b890f7a029d92fd3@10.150.0.71:30301,
enode://068a492dfa91f60d8d728fc3d594f51a08571ab3b95fe71e1b9bd939c0e251d23820580af87753017dd
644914e0cfccfc872c857030ca3ba6a4739e414ca7360@10.153.0.71:30301,
enode://e26fc5389f27e4adb24f859c4739fc6cf7e95ef008ad491cb66d0fbc144c7815c60d956bf95518842ef
35dcf4f48842e84cc614e0f6c0b404683f2fbb9d7aa45@10.161.0.71:30301,
enode://9a9be600a8ca078c96ded6f44b59ac9ab450fa61c4d5de47a1a1d6bfea7315f1b7e4fa24ef1c7ab5f67
149e4fd0b84a922615713c90b01d705479afd4b169c6b@10.164.0.71:30301,
enode://763815de0d0751de72cf136df6139cb656f7c32ea07988455c9fa52e0f01544ba10e5d249ee5d1e417a
a4b542d5d0e15e9da03cea0b3f821b890f7a029d92fd3@10.150.0.71:30301,
```

```
root@25f8b260094b / # ls -la /eth-genesis.json
-rwxrwxrwx 1 1000 1000 3947 Jun 10 07:11 /eth-genesis.json
```

执行配置指令，结果如下，启动成功：

```
INFO [06-12|11:43:44.819] Imported new chain segment      blocks=2048 txs=2  mgas=0
.042 elapsed=180.831ms mgasps=0.232 number=2432 hash=29bc1a..a39060 age=6h34m9s  dirty=3.0
8KiB
INFO [06-12|11:43:44.932] Imported new chain segment      blocks=1556 txs=0  mgas=0
.000 elapsed=109.596ms mgasps=0.000 number=3988 hash=b4508d..165054 dirty=3.08KiB
INFO [06-12|11:43:45.566] New local node record          seq=1,781,264,624,052 id
=08e8c4b5f27d17bc ip=10.150.0.74 udp=30303 tcp=30303
INFO [06-12|11:43:47.629] Imported new chain segment      blocks=2    txs=0  mgas=0
.000 elapsed="380.803µs" mgasps=0.000 number=3990 hash=58d057..f4ba52 dirty=3.08KiB
INFO [06-12|11:43:53.005] Imported new chain segment      blocks=1    txs=0  mgas=0
.000 elapsed="106.836µs" mgasps=0.000 number=3991 hash=5f9862..13cee7 dirty=3.08KiB
■
```

配置完成后执行以下任务：

- 在此节点上执行 "geth attach" 命令以获得 JavaScript 控制台，接着使用 admin.peers 命令查看当前节点连接的对等节点列表。

To exit, press ctrl-d or type exit

> admin.peers

```
[{
  caps: ["eth/66", "eth/67"],
  enode: "enode://45d8bfb981db520b5b30cd4739a86926b299bac2acb7ffd9b1bc70dd500202e8177592
340c0dca264410cd8f81bf7a8318897a4c9acc4474d8c30c0da4f2bab@10.153.0.71:43750",
  id: "0fda2b4243dbca75bb29a2636b45efe37f79d2a647e7dad49a08292ecd94bbff",
  name: "Geth/NODE_4/v1.10.26-stable-e5eb32ac/linux-amd64/go1.18.10",
  network: {
    inbound: true,
    localAddress: "10.150.0.74:30303",
    remoteAddress: "10.153.0.71:43750",
    static: false,
    trusted: false
  },
  protocols: {
    eth: {
      difficulty: 7974,
      head: "0x58d05766824c76014395b0970464cefdc5eeaf11744ce2769c72764edf4ba52",
      version: 67
    }
  }
},
]
```

可以看到已经连接到很多同等节点。

- 在同一控制台中，执行 personal.newAccount() 命令创建一个新账户，将这个新创建的账户称之为账户 Z。

```
> personal.newAccount("password123")
"0xbb7b0f570ae821c13aeef53695c62df9d692e05a"
> eth.accounts
["0xbb7b0f570ae821c13aeef53695c62df9d692e05a"]
>
```

以密码 password123 创建账户 Z，地址为 0xbb7b0f570ae821c13aeef53695c62df9d692e05a。

- 修改任务2中的 Python 代码 (web3_raw_tx.py)，连接到这个新节点，并通过这个新节点向账户 Z 发送交易，转一些以太币给账户 Z。修改后代码如下：

```
#!/bin/env python3
from web3 import Web3
from eth_account import Account

web3 = Web3(Web3.HTTPProvider('http://10.150.0.74:8545'))

# Sender's private key
key = '72c28c0d980b5e26435fc7eb8afaa27a5a117359669d73284f69ed8a401c6a85'
```

```
sender = Account.from_key(key)

recipient =
web3.toChecksumAddress('0xF5406927254d2dA7F7c28A61191e3Ff1f2400fe9')

tx = {
    'chainId': 1337,
    'nonce': web3.eth.getTransactionCount(sender.address),
    'from': sender.address,
    'to': recipient,
    'value': web3.toWei("11", 'ether'),
    'gas': 200000,
    'maxFeePerGas': web3.toWei('4', 'gwei'),
    'maxPriorityFeePerGas': web3.toWei('3', 'gwei'),
    'data': ''
}

# Sign the transaction and send it out
signed_tx = web3.eth.account.sign_transaction(tx, sender.key)
tx_hash = web3.eth.sendRawTransaction(signed_tx.rawTransaction)

# Wait for the transaction to appear on the blockchain
print("Transaction sent, waiting for receipt ...")
tx_receipt = web3.eth.wait_for_transaction_receipt(tx_hash)
print("Transaction Receipt: {}".format(tx_receipt))
```

运行结果如下：

```
[06/12/26]seed@VM:~/.../Files$ python3 web3_raw_tx.py
/usr/lib/python3/dist-packages/requests/__init__.py:89: RequestsDependencyWarning: urllib3
(2.2.3) or chardet (3.0.4) doesn't match a supported version!
warnings.warn("urllib3 ({}) or chardet ({}) doesn't match a supported "
Transaction sent, waiting for receipt ...
Transaction Receipt: AttributeDict({'blockHash': HexBytes('0x37e41ec4b31f956c38d3c934c2eeb
e50fb0b9197d2d42205c996ec14464d681c'), 'blockNumber': 4045, 'contractAddress': None, 'cumula
tiveGasUsed': 21000, 'effectiveGasPrice': 3000000007, 'from': '0xF5406927254d2A7F7c28A6119
1e3Ff1f2400fe9', 'gasUsed': 21000, 'logs': [], 'logsBloom': HexBytes('0x000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000000000000000000
e3Ff1f2400fe9', 'transactionHash': HexBytes('0x929e8f564e601af36ce48e5557f0367ae5998b9c0f24
72dc9c066a42d2d072b2'), 'transactionIndex': 0, 'type': '0x2'})
```

可以看到有发送记录，同时账户内容变化：

Recent Blocks	Recent Transactions
4059 : 0x462ef2...6e1b3a (1)	4059: 0xe23f...d43f (0xF54...fe --> 0xBB7...05)
4058 : 0x032e31...dbef63 (0)	4045: 0x929e...072b (0xF54...fe --> 0xF54...fe)
4057 : 0x404111...11...00 (0)	

```
> web3.fromWei(eth.getBalance("0xBB7B0f570ae821c13AEEF53695c62dF9D692E05a"), "ether")
11
```

- 在 JavaScript 控制台中，从账户 Z 发送交易到另一个账户。为了发送要先解锁账户 Z，然后发送指令，查看结果如下：

```
> personal.unlockAccount("0xBB7B0f570ae821c13AEEF53695c62dF9D692E05a", "password123")
true
> var target = "0xF5406927254d2dA7F7c28A61191e3Ff1f2400fe9"
undefined
> var amount = web3.toWei(5, "ether")
undefined
>
> eth.sendTransaction({from: "0xBB7B0f570ae821c13AEEF53695c62dF9D692E05a", to: target, value: amount})
"0x29d95f3348f30a796e637080f92cddbffd876c2fb5fe610cdeb3b8ea15710591"
> web3.fromWei(eth.getBalance("0xBB7B0f570ae821c13AEEF53695c62dF9D692E05a"), "ether")
5.9999789999999853
> web3.fromWei(eth.getBalance("0xF5406927254d2dA7F7c28A61191e3Ff1f2400fe9"), "ether")
13.1998109999999559
```

victim 原本是 19 左右，因此可以看出发送成功了。